Universidad Externado de Colombia | DEPARTAMENTO DE MATEMÁTICAS

Ciencia de Datos

Estructura de Datos
Parcial II
19 de octubre de 2026

Nombre:	*Documento:*

- Durante el examen no se responden preguntas y tiene 1 hora y 45 minutos para resolverlo.
- No puede usar aparatos electrónicos y el examen es individual.
- **Por favor disfrutar el examen y tener confianza en sus conocimientos**
- Tenga en cuenta: *"Qué hubiera sido, si antes ... "* Dario Gómez

Ejercicio 1. (1 punto) Responder los siguientes ejercicios sobre la clase *SinglyLinkedList*:

✓ **(0.5 puntos)** Escribir una función *maximo(L)* que reciba una *SinglyLinkedList L* y retorne el **máximo** de sus elementos.

```
def maximo(L):
    ...
```

✓ **(0.5 puntos)** Agregue a la clase *SinglyLinkedList* un método *contiene(self, x)* que retorne *True* si el elemento *x* se encuentra en la lista y *False* en caso contrario.

```
def contiene(self, x):
    ...
```

Ejercicio 2. (2 puntos) Responder los siguientes ejercicios sobre árboles binarios:

✓ **(1 punto)** Escriba una función *nodos_con_nieto_par(T)* que reciba un árbol binario *T* (*LinkedBinaryTree*) cuyos nodos almacenan números enteros y retorne el **número de nodos** que tienen **al menos un nieto par**, es decir, al menos un hijo de alguno de sus hijos almacena un número par.

```
def nodos_con_nieto_par(T):
    ...
```

✓ **(1 punto)** Dibuje un árbol binario *T* que satisfaga simultáneamente lo siguiente:

- Cada nodo de *T* almacena un único carácter.
- Un recorrido *preorder* de *T* produce `ARBOLES`.
- Un recorrido *inorder* de *T* produce `BORAELS`.

Ejercicio 3. (2 puntos) Responder los siguientes ejercicios sobre colas de prioridad:

✓ **(1 punto)** Implemente la clase *PQStack*, una pila (*Stack*) construida usando **únicamente** una cola de prioridad (*HeapPriorityQueue*) y **una** variable entera adicional.

```
class PQStack:
    def __init__(self):
        self.datos = HeapPriorityQueue()
        ...

    def push(self, e):
        ...

    def pop(self):
        ...
```

✓ **(1 punto)** Sobre un *heap* inicialmente vacío se realizan las siguientes 7 operaciones:

```
add(6,A), add(9,B), add(3,C), add(7,D), add(1,E), remove_min(), remove_min()
```

Dibuje el *heap* después de cada operación e indique qué retorna cada *remove_min()*.

Ejercicio 4. (1 punto) Considere la función hash $h(i) = (5i + 3) \bmod 11$ y una tabla hash de **11 entradas**. Se insertan, en este orden, las siguientes llaves (ya se calculó el valor de $h(i)$ para cada una):

Llave i	17	31	8	42	25	60	14	3	29	50
$h(i)$	0	4	10	4	7	6	7	7	5	0

✓ **(0.5 puntos)** Dibuje la tabla hash resultante suponiendo que las colisiones se manejan con **encadenamiento** (*separate chaining*).

✓ **(0.5 puntos)** Dibuje la tabla hash resultante suponiendo que las colisiones se manejan con **sondeo lineal** (*linear probing*).